



D S P 4 0 4

Unidad I

Operadores y Control de Flujo

S e m a n a 2

if · switch for · while · foreach

Operadores relacionales

Conversión de tipos

Estructuras de control

Temas de hoy:

- Operadores relacionales y lógicos
- Conversión de tipos (TryParse, Convert)
- Decisión: if / else if / else
- Switch expression (C# moderno)
- Ciclos: for, while, foreach

¿Qué lograrás al terminar esta clase?



Usas operadores relacionales y lógicos

Comparas valores y combinas condiciones con `&&` `||` `!`

Conviertes texto a número de forma segura

`int.TryParse` para no romper el programa si el usuario escribe mal

Controlas el flujo con `if` / `else if` / `else`

El programa toma decisiones según condiciones

Simplificas decisiones con `switch expression`

Sintaxis moderna de C# para múltiples casos

Repites código con `for`, `while` y `foreach`

Tres formas distintas de iterar — cada una para su caso

Bloque 1

Operadores Relacionales y Lógicos

Comparar valores y combinar condiciones

Operadores relacionales — ¿qué son y para qué sirven?

 **Un operador relacional compara dos valores y devuelve una respuesta de SÍ o NO (true o false).**

Sin comparaciones, un programa no puede tomar decisiones. Toda instrucción if, while o for depende de un operador relacional.

Op	Pregunta que hace	Ejemplo	Resultado	Caso real
<code>==</code>	¿Son iguales?	<code>10 == 10</code>	true	<code>nota == 7.0</code>
<code>!=</code>	¿Son distintos?	<code>5 != 3</code>	true	<code>estado != 'A'</code>
<code>></code>	¿Mayor que?	<code>8 > 5</code>	true	<code>edad > 18</code>
<code><</code>	¿Menor que?	<code>3 < 7</code>	true	<code>precio < 100</code>
<code>>=</code>	¿Mayor o igual?	<code>5 >= 5</code>	true	<code>nota >= 6.0</code>
<code><=</code>	¿Menor o igual?	<code>4 <= 9</code>	true	<code>stock <= 0</code>

Operadores lógicos — combinar condiciones

Los operadores lógicos permiten combinar varias condiciones en una sola expresión.

Ejemplo real: el descuento aplica SI el cliente es frecuente Y el monto es mayor a \$50.

& &

AND — Y

Devuelve true SOLO si AMBAS condiciones son verdaderas.

Ejemplo:

```
edad >= 18 && tieneCarne == true
```

Tabla de verdad:

A	B	R
T	T	T
T	F	F
F	T	F
F	F	F

| |

OR — O

Devuelve true si AL MENOS UNA condición es verdadera.

Ejemplo:

```
esSocio || montoCompra >= 100
```

Tabla de verdad:

A	B	R
T	T	T
T	F	T
F	T	T
F	F	F

!

NOT — Negación

Invierte el valor: true → false, false → true.

Ejemplo:

```
!estaLogueado
```

Tabla de verdad:

A	R
T	F
F	T

Operadores relacionales y lógicos — en código

CS

```
int edad = 20;
double nota = 7.5;
bool esSocio = true;

// — Relacionales —
Console.WriteLine(edad >= 18);           // True
Console.WriteLine(nota == 10.0);         // False
Console.WriteLine(edad != 15);           // True

// — Lógicos —
bool puedeVotar    = edad >= 18 && nota > 0;    // True
bool obtienePremio = nota >= 9.0 || esSocio;    // True (esSocio=true)
bool necesitaApoyo = !(nota >= 6.0);           // False (7.5 >= 6.0)

Console.WriteLine($"¿Puede votar?    {puedeVotar}");
Console.WriteLine($"¿Premio?        {obtienePremio}");
Console.WriteLine($"¿Necesita apoyo? {necesitaApoyo}");

// — Combinados con paréntesis —
bool aplicaDescuento = esSocio && (nota >= 8.0 || edad < 25);
Console.WriteLine($"¿Aplica descuento? {aplicaDescuento}"); // True
```



El resultado de cualquier expresión relacional o lógica es siempre bool: true o false. Puedes guardarlo en una variable bool.

Bloque 2

Conversión de Tipos Segura

`int.TryParse` — nunca más un programa roto

Conversión segura — ¿por qué int.Parse puede fallar?

🚫 El problema con int.Parse:

Si el usuario escribe 'abc' o deja el campo vacío, int.Parse lanza una excepción y el programa se detiene abruptamente. Eso es inaceptable en una app real.

💡 Regla práctica: usa int.Parse solo cuando SABES que el dato viene de tu propio código. Si viene del usuario → siempre TryParse.

✖ int.Parse — peligroso

Usuario escribe

"25" → int 25 ✅

"abc" → 💣 EXCEPCIÓN

Programa se detiene

Usuario ve pantalla roja

✅ int.TryParse — seguro

Usuario escribe

"25" → éxito=true, n=25 ✅

"abc" → éxito=false, n=0

Programa muestra mensaje

Sigue funcionando 🎉

int.TryParse y Convert — sintaxis completa

CS

```
// — int.TryParse —————  
Console.Write("Ingresa tu edad: ");  
string entrada = Console.ReadLine();  
  
bool exito = int.TryParse(entrada, out int edad);  
  
if (exito)  
    Console.WriteLine($"Edad válida: {edad}");  
else  
    Console.WriteLine("❌ Eso no es un número entero.");  
  
// — La versión corta (in-line out) —————  
if (int.TryParse(Console.ReadLine(), out int nota))  
    Console.WriteLine($"Nota: {nota}");  
else  
    Console.WriteLine("Entrada inválida.");  
  
// — Convert - variantes por tipo —————  
int    n = Convert.ToInt32("42");  
double d = Convert.ToDouble("3.14");  
bool   b = Convert.ToBoolean("true");  
string s2= Convert.ToString(99);      // 99 → "99"  
  
// — double.TryParse para decimales —————  
if (double.TryParse("8.75", out double promedio))  
    Console.WriteLine($"Promedio: {promedio:F1}"); // 8.8
```

Bloque 3

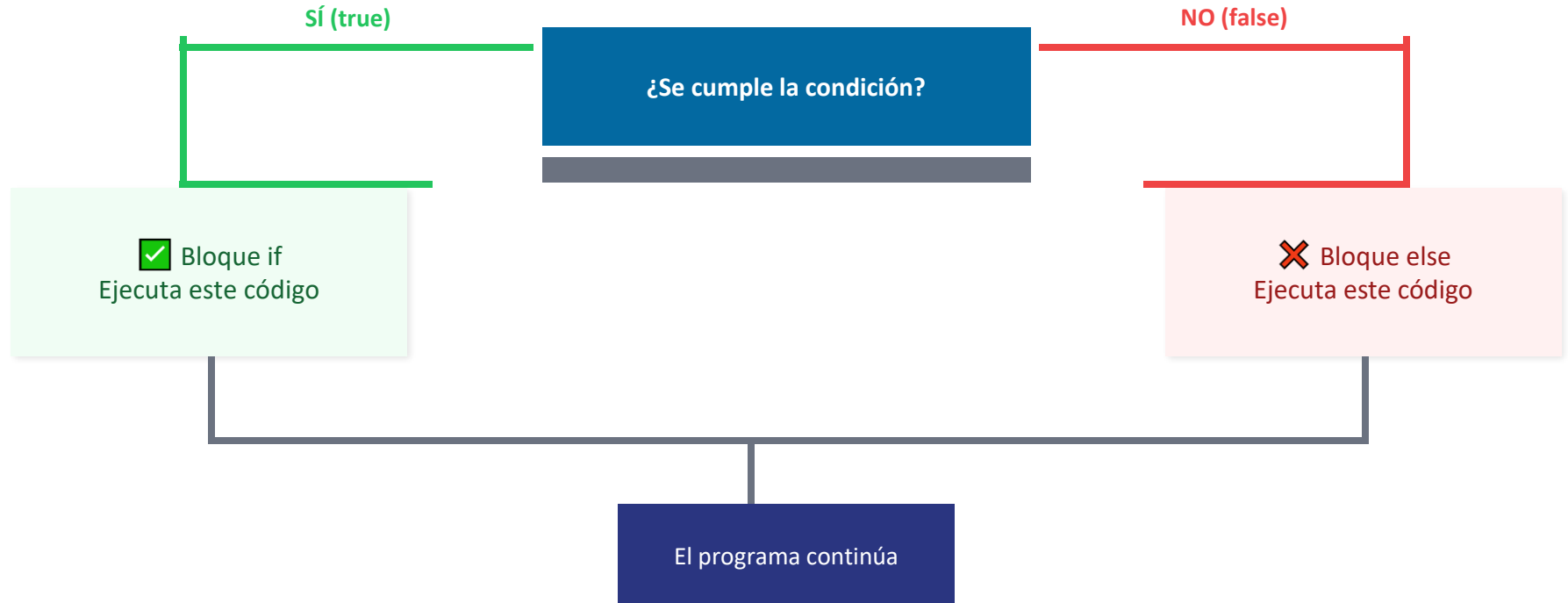
Estructuras de Decisión if · else if · else

El programa aprende a elegir

¿Qué es una estructura if? — el concepto



Un if es una instrucción que dice: 'SI se cumple esta condición, ejecuta este bloque de código; DE LO CONTRARIO, ejecuta otro'.



if / else if / else — sintaxis y variantes

CS

```
// — Forma 1: solo if —————
int edad = 20;
if (edad >= 18)
    Console.WriteLine("Es mayor de edad.");

// — Forma 2: if / else —————
double nota = 5.5;
if (nota >= 6.0)
    Console.WriteLine("Aprobado 🟢");
else
    Console.WriteLine("Reprobado 🔴");

// — Forma 3: if / else if / else (múltiples ramas) —————
double promedio = 8.2;

if (promedio >= 9.0)
    Console.WriteLine("Excelente 🏆");
else if (promedio >= 7.0)
    Console.WriteLine("Muy bien 👍");
else if (promedio >= 6.0)
    Console.WriteLine("Aprobado 🟢");
else
    Console.WriteLine("Reprobado 🔴");

// — Forma 4: if con bloque { } para múltiples instrucciones —————
if (promedio >= 6.0) {
    Console.WriteLine("Felicidades, pasaste.");
    Console.WriteLine($"Tu promedio fue: {promedio:F1}");
}
```

Reglas importantes

()

La condición va entre paréntesis

if (edad >= 18) — obligatorio

{ }

Llaves para bloques de código

Si hay más de una línea, son obligatorias



Solo una rama se ejecuta

Cuando una es true, el resto se ignora



Cuidado con = dentro del if

if (x = 5) compila pero no compara



C# no tiene 'elsif' como otros lenguajes

Switch Expression (C# moderno)

Múltiples casos sin escribir 100 if

¿Qué es switch y cuándo usarlo?



switch compara una sola expresión contra múltiples valores posibles.

Cuando tienes muchos 'else if' sobre la misma variable, switch hace el código más limpio y legible.



Con if/else if (repetitivo):

CS

```
if (dia == 1)
    Console.WriteLine("Lunes");
else if (dia == 2)
    Console.WriteLine("Martes");
else if (dia == 3)
    Console.WriteLine("Miércoles");
else if (dia == 4)
    Console.WriteLine("Jueves");
```

Usa switch cuando...

- Comparas la misma variable contra muchos valores
- Los valores son exactos (números o strings)
- Quieres código limpio y legible



Con switch expression (limpio):

CS

```
string nombre = dia switch
{
    1 => "Lunes",
    2 => "Martes",
    3 => "Miércoles",
    4 => "Jueves",
    5 => "Viernes",
    _ => "Fin de semana"
};
Console.WriteLine(nombre);
```

Usa if/else cuando...

- Las condiciones son distintas (distintas variables)
- Necesitas rangos (mayor que, menor que)
- Combinas condiciones con && o ||

Switch — sintaxis clásica y switch expression (C# moderno)

CS

```
// — SINTAXIS CLÁSICA (compatible con todo C#) —————
int opcion = 2;
switch (opcion)
{
    case 1:
        Console.WriteLine("Suma");
        break;
    case 2:
        Console.WriteLine("Resta");
        break;
    case 3:
    case 4:
        Console.WriteLine("Multiplicación o División"); // casos combinados
        break;
    default:
        Console.WriteLine("Opción no válida");
        break;
}

// — SWITCH EXPRESSION (C# 8+ — forma moderna y concisa) —————
string resultado = opcion switch
{
    1 => "Suma",
    2 => "Resta",
    3 or 4 => "Multiplicación o División",
    _ => "Opción no válida" // _ es el caso default
};
Console.WriteLine(resultado);

// — CON STRINGS —————
string día = "lunes";
string tipo = día switch
{
    "lunes" or "martes" or "miércoles" or "jueves" or "viernes" => "Día hábil",
    "sábado" or "domingo" => "Fin de semana",
    _ => "Día desconocido"
};
```

Diferencias clave

	Clásico	Expression
break	Obligatorio	No necesita
Asignar	Extra código	Directo
Versión	Todos	C# 8+
Legibilidad	Regular	Alta

★ Para proyectos nuevos en C# 10+,
prefiere switch expression — es más
conciso y evita olvidar el break.

Bloque 5

Estructuras Repetitivas for · while · foreach

Cuando necesitas hacer lo mismo muchas veces

¿Qué es un ciclo (loop)? — el concepto



Un ciclo permite ejecutar un bloque de código varias veces sin tener que escribirlo repetido.

Sin ciclos tendrías que escribir 100 `Console.WriteLine` para imprimir 100 números. Con un ciclo, lo haces en 3 líneas.

for

Úsalo cuando:

Sabes cuántas veces va a repetir

Ejemplos:

- Imprimir números del 1 al 10
- Recorrer un arreglo
- Tabla de multiplicar

En JS ya lo viste:

```
for (let i=0; i<10; i++)
```

while

Úsalo cuando:

No sabes cuántas veces — repite
MIENTRAS se cumpla algo

Ejemplos:

- Pedir dato hasta que sea válido
- Menú hasta que el usuario salga
- Esperar una condición

En JS ya lo viste:

```
while (condicion) { }
```

foreach

Úsalo cuando:

Quieres recorrer todos los elementos de
una colección

Ejemplos:

- Recorrer lista de alumnos
- Procesar cada elemento
- Imprimir todos los items

En JS ya lo viste:

```
array.forEach(item => )
```

Ciclo for — anatomía, sintaxis y ejemplos

```
for ( int i = 0 ; i < 10 ; i++ )
```

Inicialización

Se ejecuta UNA vez al inicio

Condición

Se verifica antes de cada vuelta

Actualización

Se ejecuta al FINAL de cada vuelta

```
// — FOR básico: 1 al 10 —
for (int i = 1; i <= 10; i++)
    Console.WriteLine($"{i} ");
// Salida: 1 2 3 4 5 6 7 8 9 10

// — FOR hacia atrás —
for (int i = 10; i >= 1; i--)
    Console.WriteLine($"{i} ");

// — FOR de 2 en 2 —
for (int i = 0; i <= 20; i += 2)
    Console.WriteLine($"{i} ");
// Salida: 0 2 4 6 8 10 12 14 16 18 20

// — Tabla de multiplicar del 5 —
for (int i = 1; i <= 10; i++)
    Console.WriteLine($"5 × {i} = {5 * i}");

// — FOR anidado: tabla de multiplicar completa —
for (int fila = 1; fila <= 3; fila++) {
    for (int col = 1; col <= 3; col++)
        Console.Write($"{fila*col,4}");
    Console.WriteLine();
}
```

CS

Ciclo while — cuando no sabes cuántas veces repetir

while repite MIENTRAS una condición sea verdadera. No necesitas saber de antemano cuántas vueltas dará. Se usa cuando el número de repeticiones depende del usuario o de un dato externo.

```
// — WHILE básico —
int contador = 1;
while (contador <= 5) {
    Console.WriteLine($"Vuelta {contador}");
    contador++;
    // ⚠️ olvidar esto = ciclo infinito
}

// — Caso real: pedir un número válido (entre 1 y 10) —
int numero = 0;
while (numero < 1 || numero > 10) {
    Console.Write("Ingresa un número entre 1 y 10: ");
    int.TryParse(Console.ReadLine(), out numero);
    if (numero < 1 || numero > 10)
        Console.WriteLine("❌ Fuera de rango, intenta de nuevo.");
}
Console.WriteLine($"Número aceptado: {numero}");

// — DO-WHILE: ejecuta AL MENOS UNA vez —
string respuesta;
do {
    Console.Write("¿Quieres continuar? (s/n): ");
    respuesta = Console.ReadLine()?.ToLower();
} while (respuesta != "s" && respuesta != "n");
Console.WriteLine($"Respondiste: {respuesta}");
```

CS

while vs do-while

	while	do-while
1ª ejecución	Solo si true	Siempre
Verificación	Al inicio	Al final

 **Ciclo infinito:** si la condición nunca se vuelve false, el programa se cuelga. Asegúrate de que el contador o la condición cambien dentro del while.

Ciclo foreach — recorrer colecciones de forma elegante

foreach recorre automáticamente cada elemento de una colección (array, lista). No necesitas un contador ni preocuparte por el índice. Es la forma más clara de leer 'todos los elementos'.

```
// — Comparación: for vs foreach —  
string[] nombres = { "Ana", "Luis", "María", "Carlos" };
```

CS

```
// Con for (necesitas índice):  
for (int i = 0; i < nombres.Length; i++)  
    Console.WriteLine($"Alumno {i+1}: {nombres[i]}");
```

```
// Con foreach (más limpio, más legible):  
foreach (string nombre in nombres)  
    Console.WriteLine($"Hola, {nombre}!");
```

```
// — Con int[] —  
int[] notas = { 8, 6, 9, 5, 10, 7 };  
int suma = 0;
```

```
foreach (int nota in notas)  
    suma += nota;
```

```
double promedio = (double)suma / notas.Length;  
Console.WriteLine($"Promedio: {promedio:F2}");
```

```
// — Con List<T> (colección dinámica) —  
var productos = new List<string> { "Laptop", "Mouse", "Teclado" };  
productos.Add("Monitor");
```

```
foreach (var producto in productos)  
    Console.WriteLine($"• {producto} ");
```

¿Cuándo NO usar foreach?



Necesitas el índice (posición)

Usa for → nombres[i]



Modificar la colección mientras iteras

Usa for inverso



Saltar elementos específicos

Usa for con condición en i



Regla fácil: si solo necesitas leer los elementos uno a uno → foreach. Si necesitas modificar la colección, usa for.

Resumen: ¿cuándo usar cada ciclo?

Guía rápida de decisión

for

Número de repeticiones conocido de antemano

```
for (init; condición; actualización)
```

- 1 al 100
- Tabla de multiplicar
- Recorrer array con índice

do-while

Igual que while, pero ejecuta AL MENOS UNA vez

```
do { } while (condición);
```

- Mostrar menú al menos una vez
- Pedir confirmación
- Jugar al menos una ronda

while

Repetir MIENTRAS algo sea verdadero (puede ser 0 veces)

```
while (condición) { }
```

- Validar entrada del usuario
- Menú hasta salir
- Esperar evento

foreach

Recorrer TODOS los elementos de una colección

```
foreach (tipo item in colección)
```

- Recorrer lista de alumnos
- Sumar array de notas
- Mostrar productos



Calculadora con Menú

Ejercicio explicado — Calculadora con menú switch + ciclo while

CS

```
// CALCULADORA CON MENÚ — DSP404 Semana 2
bool continuar = true;

while (continuar) {
    // — Menú —
    Console.WriteLine("
=== CALCULADORA ===");
    Console.WriteLine("1. Suma      2. Resta");
    Console.WriteLine("3. Multiplicación  4. División");
    Console.WriteLine("0. Salir");
    Console.Write("Elige una opción: ");

    if (!int.TryParse(Console.ReadLine(), out int opcion)) {
        Console.WriteLine("❌ Opción inválida."); continue;
    }
    if (opcion == 0) { continuar = false; break; }

    // — Leer operandos (con validación) —
    Console.Write("Número A: ");
    if (!double.TryParse(Console.ReadLine(), out double a)) {
        Console.WriteLine("❌ Número inválido."); continue;
    }
    Console.Write("Número B: ");
    if (!double.TryParse(Console.ReadLine(), out double b)) {
        Console.WriteLine("❌ Número inválido."); continue;
    }

    // — Switch expression para calcular —
    string resultado = opcion switch {
        1 => $"{a} + {b} = {a + b}",
        2 => $"{a} - {b} = {a - b}",
        3 => $"{a} × {b} = {a * b}",
        4 when b != 0 => $"{a} / {b} = {a / b:F4}",
        4 => "❌ No se puede dividir por cero.",
        _ => "Opción no válida."
    };

    Console.WriteLine($"Resultado: {resultado}");
}
Console.WriteLine(";Hasta luego!");
```



¡Tu turno! — FizzBuzz Mejorado

30 minutos | Individual o en parejas
Lee el enunciado en el siguiente slide



EJERCICIO — FizzBuzz Mejorado + Clasificador



Parte 1 — FizzBuzz (for)

Imprime los números del 1 al 50 usando un ciclo for.

- Si el número es divisible entre 3 → imprime 'Fizz'
- Si el número es divisible entre 5 → imprime 'Buzz'
- Si es divisible entre 3 Y 5 → imprime 'FizzBuzz'
- Si no es ninguno de los anteriores → imprime el número
- Al final muestra cuántos 'FizzBuzz' encontraste

Parte 2 — Clasificador (switch + while)

Pide al usuario un número con while hasta que ingrese 0.

Para cada número, clasifícalo con switch expression:

- Es 7 → 'Número de la suerte'
- Es par → 'Par' (usa `% 2 == 0`)
- Es impar → 'Impar'

Al terminar (0) muestra cuántos números ingresó



Pistas:

- En FizzBuzz: verifica primero el caso 15 (divisible por 3 Y 5), luego los casos solos
- Divisible: usa el operador % → si `numero % 3 == 0` es divisible entre 3
- Para contar, declara un `int contador = 0` antes del ciclo y haz `contador++` cuando corresponda

Errores comunes — Semana 2

Los errores que TODOS cometen al aprender control de flujo

= en vez de == dentro del if

```
if (edad = 18) // ❌ Asignación, no
comparación
```

```
if (edad == 18) // ✅
```

 *'Cannot implicitly convert type int to bool'*

Olvidar break en switch clásico (fall-through)

```
case 1:
    Console.WriteLine("A");
    // ❌ sin break: cae al case 2
```

```
case 1:
    Console.WriteLine("A");
    break; // ✅
```

 *No hay error de compilación — pero el comportamiento es inesperado*

Ciclo infinito en while — olvidar actualizar el contador

```
int i = 0;
while (i < 10)
    Console.WriteLine(i); // ❌ i nunca
cambia
```

```
int i = 0;
while (i < 10) {
    Console.WriteLine(i);
    i++; // ✅
```

 *El programa se cuelga — usa Ctrl+C para detenerlo*

Resumen — Semana 2

✓ `== != > < >= <=` — Operadores relacionales — siempre devuelven bool (true/false)

✓ `&& || !` — AND = ambas verdaderas. OR = al menos una. NOT = invierte.

✓ `int.TryParse` — Convierte texto a número SIN lanzar excepción si el texto es inválido.

✓ `if / else if / else` — Ejecuta un bloque según la condición. Solo una rama se ejecuta.

✓ `switch expression` — Compara un valor contra casos. Más limpio que múltiples if. `_` = default.

✓ `for` — Número fijo de repeticiones. init; condición; actualización.

✓ `while / do-while` — Repite mientras condición sea true. do-while ejecuta al menos una vez.

✓ `foreach` — Recorre cada elemento de un array o lista. Sin índice, sin contador.

Próxima clase — Semana 3: Arrays unidimensionales, multidimensionales y `List<T>`

Para la próxima clase — Guía de ejercicios

 Guía de ejercicios graduada (ver documento adjunto) — entrega para la próxima clase

E1 (Fácil)

Par o impar — if simple

E2 (Fácil)

Clasificar temperatura — if/else if/else

E3 (Fácil)

Día de la semana — switch

E4 (Medio)

Suma de 1 a N — for con TryParse

E5 (Medio)

Contador de pares e impares — for

E6 (Medio)

Número mayor — while con validación

E7 (Difícil)

Tabla de multiplicar formateada — for anidado

E8 (Difícil)

Mini cajero ATM — while + switch + TryParse